

os第一次理论作业

1.

什么是多道程序设计？多道程序设计与分时系统的区别是什么？

- 多道程序设计技术，是指允许多个程序同时进入内存并运行。即同时把多个程序放入内存中（前提是内存放的下），并允许它们交替在CPU中运行，它们共享系统中的各种硬、软件资源。当一道程序因I/O请求而暂停运行时，CPU便立即转去运行另一道程序。多道程序设计技术不仅使CPU得到充分利用，同时也改善I/O设备和内存的利用率，从而提高了整个系统的资源利用率和系统吞吐量（即单位时间内处理作业（程序）的个数），最终提高了整个系统的效率。
- 分时系统的每个用户都感觉独占计算机（通过快速切换时间片实现），微观上看是各用户轮流使用计算机，宏观上看是各用户并行工作。而多道程序设计只是避免CPU空闲，并不保证响应时间。分时操作系统是给不同用户提供程序的使用，而多道程序系统则是不同程序间的切换运行。分时操作系统主要是针对于多用户来说的，而多道程序系统主要是针对于多程序来说的。

2.

什么原因推动了操作系统从批处理发展到多道程序，进而发展到分时系统？

- 操作系统发展的核心推动力来自提升资源利用率和改善人机交互体验的需求。
- 早期批处理系统中CPU在I/O等待时完全空闲，而内存管理技术（如动态地址重定位）和通道控制器实现I/O与计算并行等技术突破使得多道程序的出现，使CPU利用率提升至85%以上。
- 随着操作系统交互需求的爆发，时钟中断机制（如DEC PDP-1的硬件定时器）和虚拟终端技术（实现多用户接入）等技术的实现使得分时系统成为可能，这与性能指标的变化和硬件的发展也密不可分。

3.

什么是陷阱？与中断的区别是什么？什么是系统调用？

- 陷阱是由程序主动触发的**同步事件**，由CPU内部触发，用于请求操作系统服务或处理异常。比如系统调用（如文件读写）和处理异常（如除零错误、内存访问违规）。
- 中断是由硬件（外设）触发的**异步事件**，通知CPU需要处理外部事务。如键盘输入、计时器信号或设备故障。
- 系统调用是用户程序通过陷阱机制请求操作系统服务的接口。流程是：一、用户程序设置参数并触发陷阱指令（如syscall）。二、CPU切换到内核模式，执行对应的服务例程。三、返回用户模式并传递结果。

4.

判断：可移植的操作系统可以从一个系统架构移植到另外一个系统架构而无需修改。

(1) 请解释为什么构建完全可移植的 OS 是不可能的？

(2) 如果你需要设计一个高度可移植的 OS，那么请描述你需要设计的两个层次？

- 为什么建立一个完全可移植性的操作系统是不可行的：
 - 硬件差异：每种硬件平台都有其特定的指令集。操作系统需要与这些指令集互动，这就需要针对每种硬件进行特定的编程。
 - 设备驱动：不同的硬件平台可能有不同的外设和组件。为了支持所有这些设备，操作系统需要包含或能够加载适合每个设备的驱动。
 - 性能优化：为了在特定的硬件上实现最佳性能，操作系统可能需要进行特定的优化。
 - 启动和初始化过程：不同的硬件有其特定的启动和初始化过程，操作系统需要与之兼容。

- 设计一个高度可移植的操作系统时的两个高级层：

硬件抽象层(HAL)：这是操作系统与硬件之间的接口层。HAL为上层的操作系统提供了一组标准的API，这些API将具体的硬件实现细节隐藏起来。当操作系统需要与硬件交互时，它会通过HAL进行。这样，为了将操作系统移植到新的硬件平台，只需要提供一个新的HAL即可。

中间件层：在HAL和应用程序之间，操作系统可能有一个中间件层，它提供了高级的服务和API，例如文件系统、网络协议栈等。这层是硬件无关的，并为应用程序提供了统一的服务和API。通过这种方式，应用程序可以在不同的硬件平台上无缝运行，只要操作系统提供了相同的中间件服务。

通过这两个高级的抽象层，操作系统可以实现高度的可移植性，尽管不能完全达到无需任何修改就可以在任何硬件平台上运行的理想状态。建立一个完全可移植的操作系统是有挑战的，主要因为不同的硬件平台有其特定的指令集、设备和驱动需求。一个完全可移植的操作系统意味着它可以在任何硬件上运行而不需要任何修改，这在实践中是非常困难的。但是，设计一个高度可移植的操作系统是可能的，通过使用抽象层来隔离硬件特定的实现。

5.

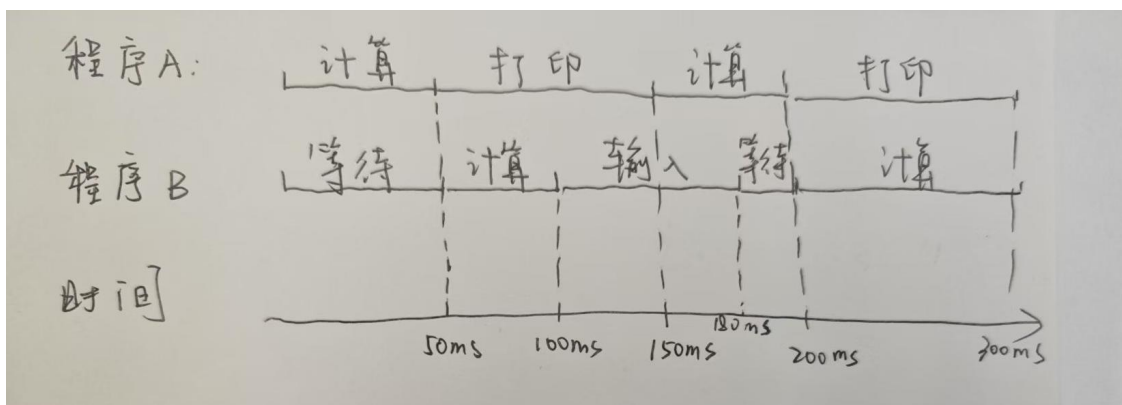
在设计操作系统时，一些设计指标是相互矛盾的，例如资源利用率、吞吐量、处理时间、健壮性等。请给出一对相互矛盾的设计实例。

- 资源利用与健壮性：在追求高资源利用率的同时，系统可能更容易受到外部干扰或错误的影响，从而降低了系统的健壮性。例如，在多任务操作系统中，为了充分利用CPU资源，可能会允许多个任务共享同一个处理器，但这样可能增加了任务间的干扰风险，降低了系统的健壮性。

6.

一个计算机系统有输入机一台、打印机两台，现有二道程序同时投入运行，且程序A先开始运行，程序B后运行。程序A的运行轨迹为：计算50ms，打印信息100ms，再计算50ms，打印信息100ms，结束。程序B运行的轨迹为：计算50ms，输入数据80ms，再计算100ms，结束。要求：

- (1) 用图画出这二道程序并发执行时的工作情况。
- (2) 说明在二道程序运行时，CPU有无空闲等待？若有，在哪段时间内等待？为什么会空闲等待？
- (3) 程序A、B运行时有无等待现象？在什么时候会发生等待现象？



- 有。100ms-150ms时间内。因为A和B都在进行IO操作
- A无。B有：0ms-50ms和180ms-200ms